chronicles_sprint2_v0

II team: Diego Bruno, Marco Crisafulli, Sebastiano Giannitti

[Link repository Github]

Introduction

Il presente documento illustra le attività di analisi, progettazione e implementazione svolte durante lo **Sprint 2**. Questa fase rappresenta il proseguimento di quanto fatto nello Sprint 1 e si pone l'obiettivo di sviluppare la parte di gestione del sonar, il rilevamento del product container nella porta di IO e l'interfacciamento hardware dei dispositivi fisici di sonar e led con il sistema.

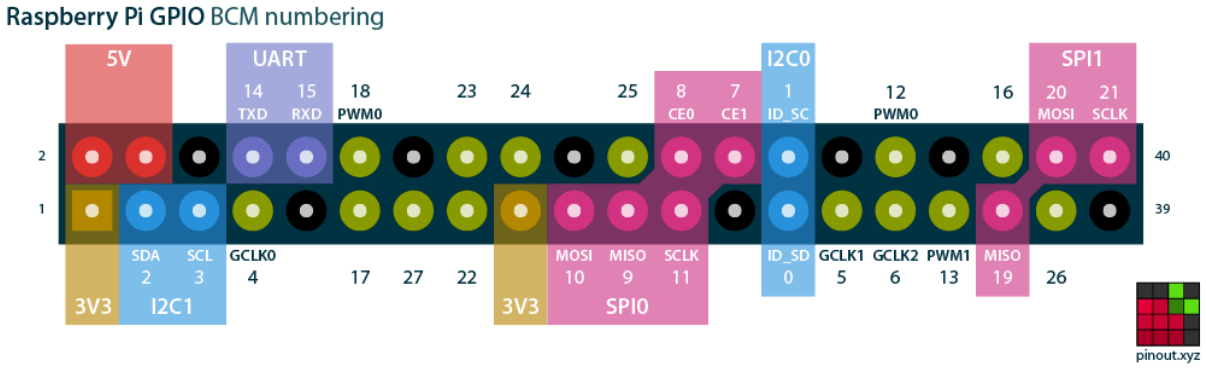
Requirements

sensor (sonar)	SNS-1	Rileva presenza container: D < DFREE/2 per ~3s ⇒ presente
sensor (sonar)	SNS-2	Guasto: D > DFREE per ≥3s ⇒ failure
LED	LED-1	Indicatore di failure sonar

sonar	Necessaria analisi del problema	In grado di rilevare la presenza di un prodotto entro una certa distanza in ioport. **Necessaria analisi del problema**
led	Necessaria analisi del problema	Viene acceso quando il sonar misura una distanza D > DFREE per almeno 3 sec. **Necessaria analisi del problema**

Problem Analysis

Da requisiti occorre realizzare un sonar che rilevi la presenza dei prodotti presso la IOport.
Il “sonar sensor” andrà posto su un dispositivo che possa supportare. La nostra software house dispone dei fantastici raspberry 3b+ che con i dovuti collegamenti supportano sia un sonar che un led.
Il Raspberry 3b+ è inizializzato con un sistema operativo basato su Ubuntu (boot da scheda sd), è in grado di connettersi alla rete wifi ed è controllato tramite ssh dal dispositivo principale su cui gira il cargoservice.
Lo schema di collegamento del sonar e led al raspberry verrà gestito internamente, riportiamo qui di seguito lo schema GPIO:



IL sonar è collegato al pin BCM 17 e 27, e alla linea di tensione da 5v.
Il led è collegato al pin bcm 25.
Gli script di basso livello che governano i componenti sopra elencati sono realizzati in python e sono disponibili sulla repo nella cartella resources del sonarLed2025. Sono implementati nel codice che governa il comportamento ad alto livello dell’intero dispositivo raspberry.
Il sonar scelto è l’HCSR04 mentre il led scelto è un HW-479. Abbiamo collegato solo il pin relativo al colore blu perchè da requisiti non è richiesto né specificato il colore del led.

Abbiamo descritto il raspberry, il sensore sonar e l’attuatore led a basso livello.
Procediamo con un’analisi comportamentale:

sonar	Necessaria analisi del problema	In grado di rilevare la presenza di un prodotto entro una certa distanza in IOport. **Necessaria analisi del problema**

Lo script python sonar.py di basso livello, stampa su stdout i valori misurati dal sonar in formato floating point.
Le dimensioni dell’ sono note:
D è la distanza misurata ogni secondo dal sonar
DFREE è la dimensione dell’IOport

Se la distanza misurata dal sonar è D < DFREE/2 allora si evince la presenza di un prodotto presso la IOport, altrimente se D > DFREE per ≥3s ⇒ failure .
Stabiliamo il valore DFREE = 10.

led	Necessaria analisi del problema	Viene acceso quando il sonar misura una distanza D > DFREE per almeno 3 secondi. **Necessaria analisi del problema**

Una volta definite le misure di D e DFREE possiamo analizzare il caso failure e quindi il caso in cui il led va acceso.
Il led è governato da 2 script di basso livello python invocati dal codice di alto livello:

- ledPythonOff25.py si occupa di spegnere il led
- ledPythonOn25.py si occupa di accendere il led

Un ciclo condizionale governerà il rilevamento del guasto "failure" a livello del raspberry.
Il sistema sonarLed2025 è scritto in linguaggio qak.

Quindi:
Disponiamo di codice di basso livello sonar.py che attiva il sonar fisico e scrive sul dispositivo standard di output, a intervalli temporali fissi, i valori della distanza corrente rilevata.

Il componente `sonardevice` è realizzato in modo esplicito, definendo un Actor nel modello applicativo. Questo attore si occuperà di interpretare i dati messi su stdout da parte di sonar.py ed emetterà uno stream locale di dati:

emitlocalstream sonardata : distance($Distance).

Un attore `mind` che si occuperà di rilevare eventuali guasti ed emettere un evento qak, mind percepisce lo stream di dati di `sonardevice`:

subscribeTo sonardevice for sonardata

Ricordando i concetti de La Clean Architecture, occorre ora rispondere alla seguente domanda: come `sonar24` intende ricevere i valori D della distanza misurata?

Per motivi di modularità ed espandibilità, si propende per l'uso di un event:

Event sonardata : distance(D)

- Eventi di questo tipo si suppongono generati da un altro componente, denominato `sonardevice`, anch'esso modellato come un Actor.

- Per motivi di efficienza e per evitare l'inutile trasmissione di eventi via rete, si ritiene opportuno utilizzare il concetto di Attori streamer, e le primitive subscribeTo e emitLocalstream.
- Il componente `sonardevice` incapsula il codice di basso livello sonar.py

La Reactive programming è una combinazione di idee riconducibili al modello Observer, al modello Iterator e al modello di programmazione funzionale.

In questo stile di programmazione, un servizio-consumatore reagisce ai dati non appena arrivano, con la capacità anche di propagare le modifiche come eventi agli osservatori registrati.

Un sistema Qak può essere impostato seguendo questo stile di programmazione emettendo eventi usando la primitiva *subscribeTo* (E.g. demostreams.qak).

Occupiamoci ora di capire come il raspberry e il sistema sonarLed2025 comunica con il cargoservice.

Il protocollo MQTT è adatto alla trasmissione di eventi nei sistemi qak:

- La dichiarazione di usare un broker MQTT avviene a livello di Sistema.
- Ogni attore implementa anche l'interfaccia org.eclipse.paho.client.mqttv3.MqttCallback. Pertanto ogni attore può gestire notifiche emesse da un MQTT client, attraverso il metodo messageArrived.

Il broker MQTT è in esecuzione sul componente principale di cargoservice e riceve eventi contenenti la distanza misurata dal sonar su un eventTopic dedicato.

Come conseguenza di una dichiarazione MQTT broker implicita, tutti i componenti del sistema cargoservice saranno in grado di recepire l'evento sonardata emesso da mind dentro il raspberry.

Questi eventi contenenti informazione vanno gestiti dal sistema cargoservice attraverso un attore dedicato per aderire ai principi de la clean architecture.
Questo attore si chiamerà **sonaradapter** e verrà "chiamato" da cargoservice prima di muovere il robot verso la IOport per verificare la presenza del container presso essa.
Abbiamo scelto di usare una request/response qak perché abbiamo bisogno di avere un esito, positivo o negativo.

```
Request sonarReq : sonarReq(X)
Reply sonarReply : sonarReply(X) for sonarReq
Reply sonarReplyFailure : sonarReplyFailure(X) for sonarReq
```

Una sonarReq verrà inviata da **cargoservice** -> **sonaradapter** e l'esito sarà positivo o negativo.
Il **sonaradpter** si preoccuperà di fare il parsing dei dati ricevuti con l'evento e interpretare la presenza di guasti o no. Sapendo che arrivano messaggi con frequenza di 1 al secondo, e avendo da requisiti la presenza di un guasto solo dopo 3 secondi di distanze > DFREE, dopo 3 messaggi si avrà la certezza di un guasto o no.
In caso di guasto verrà inviata una risposta con esito negativo al cargoservice che si preoccuperà di inviare una nuova richiesta di valutazione della distanza a **sonaradapter**.

Il sonar registra dati 1 volta al secondo e questi vengono inviati sotto forma di evento con mqtt da mind e percepiti da sonaradpter 1 volta ogni secondo. Percio' e' sufficiente una verifica alla prima ricezione dati e alla terza ricezione (approssima un tempo di 3 sec come da requisito)

1. Attendi richiesta
2. Leggi distanza e verifica se regolare
3. Aspetta 2 secondi e ricontrolla distanza
4. Se distanza ok -> reply ---- se distanza not ok -> replyFailure
5. L'attore si esegue nuovamente su nuova richiesta in caso di fallimento

Test Plans

In questa sezione definiamo la strategia di verifica adottata per validare la business logic del `cargoservice`. L'obiettivo è garantire che il sistema rispetti i requisiti funzionali e gestisca correttamente le anomalie prima di procedere all'integrazione con l'hardware reale.

Per eseguire i test in modo automatizzato e ripetibile, isolando la logica di business dalle dipendenze esterne, abbiamo predisposto un ambiente composto da:

- `ctxusermock`**:** Un contesto QAK dedicato che ospita un attore *mock* (simulatore cliente). Questo attore invia sequenze predeterminate di `requesttoload` e verifica che le risposte del sistema (`loadaccepted`/`loadrefused`) corrispondano alle attese.
- `productservice`**:** Il database è popolato con un set di prodotti noti (es. PID validi, PID inesistenti, PID con peso eccessivo) per stimolare deterministicamente le diverse ramificazioni del codice.

Sulla base degli *Edge Cases* identificati in fase di analisi , abbiamo formalizzato la seguente suite di test funzionali.

Scenario:	**Contesto:**	**Condizione del sistema**	**Risultato Atteso**
Carico presente all'IOPORT	**Cargoservice**	sonarReq(X) da **cargoservice** whenEvent sonardata D < DFREE/2 per t>3s	**sonaradpter [# GUASTO = false #]** e invio di **sonarReply : sonarReply(ok)** a **cargoservice**

	Raspberry	Distance > 0 emitlocalstream sonardata : distance($Distance) D < DFREE	**mind: emit sonardata : sonardata($D)** **python ledPython25Off.py**
D > DFREE per 3 secondi	**Cargoservice**	sonarReq(X) da **cargoservice** whenEvent sonardata D > DFREE per t>3s	**sonaradpter [# GUASTO = true #]** e invio di **sonarReply : sonarReplyFailure(X)** a **cargoservice**
	Raspberry	Distance > 0 emitlocalstream sonardata : distance($Distance) D > DFREE	**mind: emit sonardata : sonardata($D)** **python ledPython25On.py**

Project

La fase di progettazione dello Sprint 2 ha concretizzato le decisioni relative all'interfacciamento hardware, integrando la ricezione dei dati dal sonar fisico all'interno dell'architettura QAK. L'obiettivo principale è stato sviluppare la gestione del rilevamento del *product container* presso la porta di I/O e la conseguente gestione dei guasti.

Applicando i principi della *Clean Architecture*, l'architettura è stata estesa per reagire ad eventi esterni generati dall'hardware fisico, modellando un componente ponte: il `sonaradapter`.

Integrazione MQTT

L'architettura di sistema è stata configurata per supportare la comunicazione distribuita con il Raspberry Pi. A livello di dichiarazione di sistema, è stato istanziato un broker MQTT esplicito in ascolto sul topic dedicato ai dati del sensore:

```
mqttBroker "localhost" : 1883 eventTopic "sonardata"
```

Questa configurazione permette al contesto `ctxcargoholdservice` di percepire globalmente gli eventi generati dal Raspberry, formalizzati come:

```
Event sonardata : distance(D)
```

Implementazione dei Componenti

`cargoservice`

L'attore principale è stato arricchito per integrare la validazione spaziale tramite sonar prima di muovere il robot verso la porta di I/O.
Invece di sottoscriversi direttamente agli eventi MQTT, il `cargoservice` delega la validazione al `sonaradapter` tramite una `Request`. Un punto di forza dell'implementazione è la gestione esplicita del fallimento (guasto o prodotto assente) tramite un meccanismo di *retry* continuo, che mette l'attore in attesa finché l'area non torna operativa:

```
State verifySonar {
      request sonaradapter -m sonarReq : sonarReq(X)
} Transition t0 whenReply sonarReply -> move
                whenReply sonarReplyFailure -> sonarErrorRetry

State sonarErrorRetry {
      delay 200
      request sonaradapter -m sonarReq : sonarReq(X)
}
```

`sonaradapter`

Questo attore traduce lo stream continuo di eventi fisici `sonardata` in una singola risposta logica (`sonarReply` o `sonarReplyFailure`) per l'orchestratore. La logica approssima il requisito temporale di "almeno 3 secondi" attraverso una FSM ben definita:

- **`awaitingRequest`**: L'attore è inattivo finché non riceve una `sonarReq`.

- **`handleSonarData`**: Al primo evento `sonardata` intercettato, estrae la distanza.
    - Se $D > DFREE$ imposta il flag `GUASTO = true`.

    - Se $D \leq DFREE/2$ imposta `GUASTO = false`.
    - Dopodiché, esegue un `delay 2000` per attendere l'evoluzione fisica della misurazione.

- **`verifyData`**: Cattura il successivo evento `sonardata` (post-delay) per effettuare una seconda lettura di conferma.

- Utilizzando la guardia `[# !GUASTO #]`, instrada il flusso verso `productAtIOPort` (inviando `sonarReply(ok)`) o verso `handleFailure` (inviando `sonarReplyFailure(ok)`).

Sottosistema Raspberry

Per mantenere il disaccoppiamento e sfruttare il paradigma della *Reactive Programming*, la logica del Raspberry è stata divisa in due attori specializzati:

- **`sonardevice`**
  Funge da *wrapper* di basso livello. Esegue lo script Python `sonar.py`, ne cattura l'output standard e lo converte in uno stream di dati locale utilizzando la primitiva QAK `emitlocalstream sonardata: distance($Distance)`.

- **`mind`**
  Agisce come controllore a bordo del dispositivo. Si sottoscrive allo stream del sensore (`subscribe To sonardevice for sonardata`), valuta la presenza di anomalie a livello locale (comandando eventualmente l'accensione del LED fisico di *failure*) ed emette (`emit`) l'evento globale verso il resto del sistema.

Protocollo dei messaggi

Le interazioni per il sottosistema sensoriale sono state tipizzate per garantire sicurezza e separazione delle responsabilità:

- **Evento Sensoriale**
  - `Event sonardata : distance(D)`

- **Comando Validazione**
  - `Request sonarReq : sonarReq(X)`

- **Esito**
  - *Positivo* `Reply sonarReply : sonarReply(X) for sonarReq`
  - *Negativo* `Reply sonarReplyFailure : sonarReplyFailure(X) for sonarReq`

Maintenance

1. **Sprint 3**
   Realizzazione della GUI (20h/uomo)